Lecture 17: Retrieval augmented generation

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will be able to...

1. Explain why language models need external knowledge
2. Describe the RAG pipeline: retrieve, augment, generate
3. Implement a basic RAG system in Python using free, open-source tools
4. Evaluate tradeoffs between RAG and fine-tuning

Announcements

Assignment 3 due today!

Wikipedia Embeddings assignment is due **today, Friday, February 6 at 11:59 PM EST**.

Submit via pull request after accepting assignment in GitHub Classroom.

Assignment 4 released

Customer Service Chatbot — build a context-aware chatbot using retrieval and generation techniques.

Due: **February 16 at 11:59 PM EST**.

The knowledge problem

Two kinds of knowledge

Parametric knowledge: facts stored *inside* the model's parameters during training. The model "knows" things because it memorized patterns from its training data.

Non-parametric knowledge: facts stored *outside* the model in documents, databases, or APIs. The model accesses this knowledge at inference time.

Why parametric knowledge falls short

1. **Hallucination**: the model confidently generates plausible-sounding but incorrect information
2. **Stale data**: knowledge is frozen at training time — the model doesn't know about recent events
3. **No citations**: the model can't tell you *where* it learned something, making claims hard to verify

What is RAG?

Retrieval Augmented Generation

RAG is a technique that gives language models access to external knowledge by *retrieving* relevant documents and *including* them in the prompt. The model generates its response based on both its parametric knowledge and the retrieved context.

Key insight

Instead of trying to store all knowledge in the model's parameters, we let the model *look things up* in a knowledge base at inference time. This separates *what the model knows how to do* (reasoning, language) from *what facts it has access to* (documents, data).

The RAG pipeline

The five stages of retrieval augmented generation

The big picture

$$\text{RAG}(q) = \text{LLM}(q + \text{retrieve}(q, \mathcal{D}))$$

where q is the query and $\mathcal{D}$ is the document collection.

Embeddings for retrieval

Connecting to lectures 11-12

You already know that embeddings map text to vectors where **semantic similarity corresponds to geometric proximity**. RAG exploits this: embed the query and documents into the same vector space, then find the documents closest to the query using cosine similarity.

Which embedding model?

For RAG, we use **sentence embedding** models (not word-level) that produce a single vector for an entire passage. All of these are free and open-source:

- **Sentence-BERT** (all-MiniLM-L6-v2): fast, good quality, 384 dimensions
- **BGE** (BAAI/bge-small-en-v1.5): state-of-the-art for retrieval
- **E5, GTE**: strong open-source alternatives

Document chunking

Breaking documents into pieces

Real documents are long. We can't embed an entire book as a single vector (too much information is lost). Instead, we split documents into smaller **chunks** and embed each chunk separately.

Chunking strategies

- **Fixed-size**: split every *n* characters/tokens (simple but may cut mid-sentence)
- **Sentence-based**: split on sentence boundaries (preserves meaning better)
- **Semantic**: use topic shifts to determine chunk boundaries (best quality but most complex)
- **Recursive**: split on paragraphs first, then sentences if chunks are still too long

Size tradeoffs

Too small = loses context. Too large = dilutes relevance. Start with **256-512 tokens** per chunk with **50-100 token overlap** between adjacent chunks.

Vector databases

Specialized storage for embeddings

A **vector database** stores embedding vectors and provides fast **approximate nearest neighbor** (ANN) search. Instead of comparing a query to every document (slow for millions of documents), vector databases use indexing structures to find the most similar vectors in milliseconds.

Why not just use a list?

Brute-force search over n vectors takes $O(n)$ time. For 10 million chunks, that's 10 million cosine similarity computations per query. Vector databases use techniques like HNSW (Hierarchical Navigable Small World graphs) to reduce this to roughly $O(\log n)$.

For this course

We'll use **ChromaDB** — no server setup, no API key, just `pip install chromadb` . For production scale, see also FAISS (Meta) and Pinecone (cloud).

Python: embed and retrieve

Steps 1-2: embed your knowledge base and find relevant documents

```python
from sentence_transformers import SentenceTransformer, util

embedder = SentenceTransformer("all-MiniLM-L6-v2")

# Knowledge base (in practice, loaded from files)
documents = [
    "The transformer was introduced by Vaswani et al. in 2017.",
    "BERT uses bidirectional attention for language understanding.",
    "GPT models use autoregressive (left-to-right) generation.",
    "Attention mechanisms allow models to focus on relevant context.",
    "Fine-tuning adapts a pre-trained model to a specific task.",
```

continued...

Python: embed and retrieve

Steps 1-2: embed your knowledge base and find relevant documents

```
]
doc_embeddings = embedder.encode(documents, convert_to_tensor=True)

# Retrieve: embed query, find nearest documents
query = "How do transformers work?"
query_emb = embedder.encode(query, convert_to_tensor=True)
scores = util.cos_sim(query_emb, doc_embeddings)[0]
top_indices = scores.argsort(descending=True)[:3]
for idx in top_indices:
    print(f"  [{scores[idx]:.3f}] {documents[idx]}")
```

...continued

Python: generation with context

Step 3: augment the prompt and generate (free, no API key)

```python
from transformers import pipeline

# Load a free, local text generation model
generator = pipeline("text2text-generation", model="google/flan-t5-base")

def rag_answer(query, documents, doc_embeddings, top_k=3):
    """Answer a question using RAG."""
    # Retrieve relevant context
    query_emb = embedder.encode(query, convert_to_tensor=True)
    scores = util.cos_sim(query_emb, doc_embeddings)[0]
    top_idx = scores.argsort(descending=True)[:top_k]
    context = "\n".join([documents[i] for i in top_idx])
```

continued...

Python: generation with context

Step 3: augment the prompt and generate (free, no API key)

```
13
14      # Build augmented prompt and generate
15      prompt = f"""Answer based on the context. Say "I don't know" if
    unsure.
16  Context: {context}
17  Question: {query}
18  Answer:"""
19      return generator(prompt, max_new_tokens=128)[0]
    ["generated_text"]
20
21  print(rag_answer("What is BERT?", documents, doc_embeddings))
```

...continued

No API key needed

FLAN-T5 runs locally in Colab. For better quality, try `google/flan-t5-large` (requires GPU).

End-to-end RAG with ChromaDB

A complete mini RAG system

```python
import chromadb
from transformers import pipeline

# Set up ChromaDB (handles embedding automatically)
client = chromadb.Client()
collection = client.create_collection("course_notes")

# Add documents
collection.add(
    documents=[
        "The transformer uses self-attention to process sequences in parallel.",
```

continued...

End-to-end RAG with ChromaDB

A complete mini RAG system

```
12          "Cross-entropy loss measures prediction error for
    classification.",
13          "RAG combines retrieval with generation for knowledge-
    grounded answers.",
14      ],
15      ids=["doc1", "doc2", "doc3"]
16  )
17
18  # Retrieve relevant documents
19  results = collection.query(query_texts=["How does attention
    work?"], n_results=2)
20  context = "\n".join(results["documents"][0])
21
22  # Generate answer using retrieved context
```

...continued...

End-to-end RAG with ChromaDB

A complete mini RAG system

```
generator = pipeline("text2text-generation", model="google/flan-t5-base")
prompt = f"Answer based on context.\nContext: {context}\nQuestion: How does attention work?\nAnswer:"
print(generator(prompt, max_new_tokens=128)[0]["generated_text"])
```

...continued

RAG vs fine-tuning

When to use which

	RAG	Fine-tuning
Knowledge updates	Easy — just update the documents	Hard — requires retraining
Cost	Cheap (no training needed)	Expensive (GPU time, data prep)
Citations	Natural — you know which docs were retrieved	Not possible
Hallucination	Reduced (grounded in retrieved text)	Can still hallucinate
Domain adaptation	Good for factual Q&A	Better for style/behavior changes
Latency	Higher (retrieval + generation)	Lower (just generation)

They're complementary

Many production systems use *both*: fine-tune the model for the domain's style and behavior, then use RAG for up-to-date factual knowledge.

Beyond basic RAG

Advanced techniques

1. **Re-ranking**: after initial retrieval, use a cross-encoder model to re-score and re-order results for better precision
2. **Hybrid search**: combine vector similarity (semantic) with keyword matching (BM25) for more robust retrieval
3. **Query expansion**: rewrite or expand the user's query using an LLM before retrieval to improve recall

Limitations to keep in mind

1. **Retrieval quality**: if the retriever fails to find relevant documents, the generator can't produce a good answer — garbage in, garbage out
2. **Context window limits**: there's a limit to how much retrieved text fits in the prompt, requiring aggressive chunking or summarization
3. **Multi-source reasoning**: RAG is great for finding a specific fact, but struggles when answers require synthesizing information across many documents

Best practices

Making RAG work well

1. **Chunk wisely**: experiment with chunk sizes (256-512 tokens is a good start). Use overlap between chunks.
2. **Choose the right embedding model**: test multiple models on your specific domain. General-purpose models may not capture domain-specific semantics.
3. **Evaluate retrieval separately**: before blaming the LLM, check if the retriever is finding the right documents.
4. **Include metadata**: store source, date, and section info with chunks so the LLM can cite sources.
5. **Handle "I don't know"**: instruct the model to say when the context doesn't contain the answer rather than hallucinating.

Try it yourself

Interactive demo

Explore our interactive RAG demo to see retrieval augmented generation in action:

RAG System Demo — Search 2,000 Wikipedia articles, configure chunking strategies, compare RAG vs. non-RAG answers, and visualize embedding spaces — all in your browser with no API keys required.

Assignment 4 connection

Your next assignment asks you to build a **Customer Service Chatbot** that uses retrieval and generation techniques. The RAG concepts from today's lecture are directly applicable!

Discussion: RAG in practice

Questions to consider

1. What kinds of applications benefit most from RAG? Where would fine-tuning be better?
2. A company wants to build a chatbot using internal documents. What are the privacy implications of RAG vs. fine-tuning?
3. Will RAG eventually be unnecessary if models get large enough to memorize everything?
4. What happens when the retrieved documents themselves contain misinformation?

References

Further reading

Lewis et al. (2020, *NeurIPS*) "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" — The original RAG paper.

Borgeaud et al. (2022, *ICML*) "Improving Language Models by Retrieving from Trillions of Tokens" — RETRO: scaling retrieval to massive corpora.

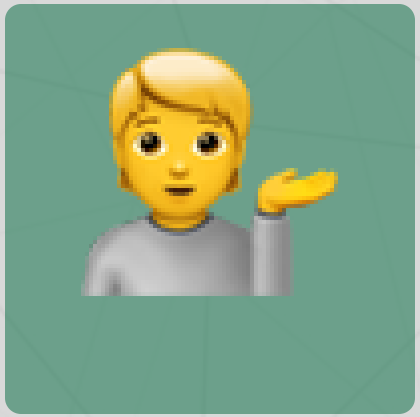
Gao et al. (2024, *arXiv*) "Retrieval-Augmented Generation for Large Language Models: A Survey" — Comprehensive survey of RAG techniques.

ChromaDB Documentation — Getting started with vector databases for RAG.

Questions?

Email

Discord



Office hours

Up next...

Week 6: BERT deep dive — bidirectional attention, masked language modeling, and the pre-train/fine-tune paradigm!